COMPLETE JAVA NOTES

For Beginners & Interview Preparation

Topic-wise • Simple Language • Interview-Ready

📦 CHAPTER 1: OOP CONCEPTS

Object-Oriented Programming (OOP)

OOP is a way of writing programs using 'objects' — just like real-world things. Instead of writing long procedures, we group data and actions together into objects.

1. Encapsulation

📌 WHAT	Wrapping data (variables) and methods (functions) together inside a class, and hiding the data from outside using private access.
❓ WHY	To protect data from being changed accidentally. Like a capsule pill — medicine is hidden inside, you can't touch it directly.
🌍 Example	A bank account — you can't directly change the balance. You must use deposit() or withdraw() methods.

💻 Code Example:

```
class BankAccount {
    private double balance;  // hidden from outside

    public void deposit(double amount) {  // controlled access
        if (amount > 0) balance += amount;
    }

    public double getBalance() {  // getter
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();
        acc.deposit(500);
        // acc.balance = 1000;  // ❌ Error! Cannot access private
        System.out.println(acc.getBalance());  // ✅ 500.0
    }
}
```

🔁 Step-by-Step Execution:

- balance is private → only accessible inside BankAccount class
- deposit() checks if amount > 0 before adding — data validation!
- getBalance() returns balance safely — read-only access

- Outside code cannot say acc.balance directly → encapsulation working!

🎯 Interview Questions:

Q1. **What is Encapsulation?**

→ Binding data and methods together and hiding data using private. Access is provided via getters/setters.

Q2. **What is the difference between Encapsulation and Abstraction?**

→ Encapsulation hides DATA (using private). Abstraction hides IMPLEMENTATION (using abstract/interface). Encapsulation = HOW data is stored. Abstraction = WHAT is done.

Q3. **Why use getters and setters?**

→ To control how data is read and written. You can add validation in setters. Direct access can corrupt data.

⚠️ Common Mistakes:

- Making variables public instead of private
- Forgetting to provide getter/setter methods
- Confusing encapsulation with abstraction in interviews

2. Inheritance

📌 WHAT	One class (child) inherits properties and methods from another class (parent). Child gets all features of parent automatically.
❓ WHY	Code reuse! Write once, use everywhere. Like a child inheriting traits from parents.
🌍 Example	Animal (parent) has eat() and sleep(). Dog (child) inherits these and also has bark().

💻 Code Example:

```
class Animal {
    String name;
    void eat() { System.out.println(name + " is eating"); }
    void sleep() { System.out.println(name + " is sleeping"); }
}

class Dog extends Animal {  // Dog inherits Animal
    void bark() { System.out.println(name + " is barking"); }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.name = "Tommy";
        d.eat();     // from Animal ✅
```

```
        d.sleep();  // from Animal ✅
        d.bark();   // Dog's own method ✅
    }
}
```

🔁 Step-by-Step Execution:

- Dog class gets all methods of Animal using 'extends' keyword
- d.eat() — Java looks in Dog, not found → looks in Animal → found → executes
- d.bark() — found in Dog → executes directly
- 'super' keyword can call parent class methods from child

🎯 Interview Questions:

Q4. **What is inheritance?**
→ Child class acquiring properties of parent class using 'extends'. Promotes code reuse.

Q5. **What is the use of 'super' keyword?**
→ Calls parent class constructor or method from child class. super() calls parent constructor, super.method() calls parent method.

Q6. **Does Java support multiple inheritance?**
→ No, Java doesn't support multiple inheritance with classes (to avoid the Diamond Problem). But it supports it through interfaces.

Q7. **What is the Diamond Problem?**
→ When class C inherits from A and B, and both A & B have same method — which one to call? Java avoids this with interfaces.

⚠️ Common Mistakes:

- Forgetting 'extends' keyword
- Trying multiple inheritance with classes (not allowed in Java)
- Not calling super() in child constructor when parent has parameterized constructor

3. Polymorphism

📌 WHAT	One thing, many forms. Same method name behaves differently based on context. 'Poly' = many, 'morph' = forms.
❓ WHY	Flexibility. One name, multiple behaviors. Like a 'draw' button — it draws circle, square, or triangle based on object.
🌍 Example	A person behaves as son at home, student in school, customer in shop — same person, different roles.

💻 Code Example (Method Overriding = Runtime Polymorphism):

```
class Shape {
    void draw() { System.out.println("Drawing shape"); }
}

class Circle extends Shape {
    @Override
    void draw() { System.out.println("Drawing circle ⭕"); }
}

class Square extends Shape {
    @Override
    void draw() { System.out.println("Drawing square ⬛"); }
}

public class Main {
    public static void main(String[] args) {
        Shape s1 = new Circle();  // parent ref, child object
        Shape s2 = new Square();
        s1.draw();  // Drawing circle ⭕
        s2.draw();  // Drawing square ⬛
    }
}
```

🔁 Step-by-Step Execution:

- s1 is of type Shape but holds Circle object — this is Upcasting
- At runtime, JVM checks: 'What is the actual object?' → Circle
- So Circle's draw() is called — decided at RUNTIME (Runtime Polymorphism)
- This is also called Dynamic Method Dispatch

🎯 Interview Questions:

Q8. **What are the types of Polymorphism?**

→ 1) Compile-time (Method Overloading) — decided at compile time. 2) Runtime (Method Overriding) — decided at runtime.

Q9. **What is Dynamic Method Dispatch?**

→ When parent class reference calls overridden method of child class at runtime. JVM decides which method to call based on actual object.

Q10. **Can polymorphism happen without inheritance?**

→ Runtime polymorphism needs inheritance/interface. Compile-time polymorphism (overloading) can happen in same class.

⚠️ Common Mistakes:

- Confusing overloading with overriding
- Forgetting @Override annotation (not mandatory but best practice)
- Thinking static methods can be overridden (they can't — they are hidden, not overridden)

4. Abstraction

📌 WHAT	Hiding internal implementation and showing only what is necessary. Focus on 'what' not 'how'.
❓ WHY	Reduces complexity. User doesn't need to know HOW it works — just WHAT it does. Like a TV remote — you press button, channel changes. You don't care how.
🌍 Example	ATM machine — you just press 'Withdraw'. You don't see the internal bank server code.

💻 Code Example (Abstract Class):

```
abstract class Vehicle {
    abstract void start();  // no body — must be implemented by child

    void stop() {  // normal method with body
        System.out.println("Vehicle stopped");
    }
}

class Car extends Vehicle {
    @Override
    void start() { System.out.println("Car starts with key 🔑"); }
}

class Bike extends Vehicle {
    @Override
    void start() { System.out.println("Bike starts with kick 🦵"); }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v = new Car();
        v.start();  // Car starts with key
        v.stop();   // Vehicle stopped
    }
}
```

🎯 Interview Questions:

Q11. **What is the difference between abstract class and interface?**

→ Abstract class: can have constructor, concrete methods, any access modifiers. Interface: no constructor, all methods public & abstract by default (before Java 8). Use abstract class for IS-A relation. Use interface for CAN-DO behavior.

Q12. **Can we instantiate an abstract class?**

→ No! Abstract class cannot be instantiated directly. You must extend it and provide implementation.

Q13. **What is an interface?**

→ A blueprint of a class. All methods are abstract by default. A class implements interface and provides method body. Used for achieving full abstraction.

⚠️ Common Mistakes:

- Trying to create object of abstract class
- Not implementing all abstract methods in child class
- Confusing interface 'implements' with class 'extends'

🏗️ CHAPTER 2: CLASS & OBJECT

Class & Object

📌 **Class**	A blueprint/template. Like a house plan — it defines what rooms, windows, doors will be there. No actual house yet.
📌 **Object**	Actual instance created from the class. Like the actual house built from the plan. It has real values.

💻 Code Example:

```
class Student {             // CLASS = blueprint
    String name;            // attributes (variables)
    int age;

    void study() {          // behavior (method)
        System.out.println(name + " is studying");
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();  // OBJECT created
        s1.name = "Rahul";
        s1.age = 20;
        s1.study();  // Rahul is studying

        Student s2 = new Student();  // Another OBJECT
        s2.name = "Priya";
        s2.study();  // Priya is studying
    }
}
```

🔁 Step-by-Step Execution:

- 'new Student()' allocates memory in Heap for the object
- s1 holds reference (address) to that memory location in Stack
- s1.name = 'Rahul' stores value in that heap memory
- Both s1 and s2 are separate objects — changes to s1 don't affect s2

🎯 Interview Questions:

Q14. **What is the difference between class and object?**

→ Class is a blueprint (template). Object is an instance of a class with real values. Class exists at compile time, Object exists at runtime.

Q15. **Where are objects stored in Java?**

→ Objects are stored in Heap memory. References (variable names) are stored in Stack memory.

Q16. **Can a class have multiple objects?**

→ Yes! You can create any number of objects from one class. Each object has its own copy of instance variables.

🔨 CHAPTER 3: CONSTRUCTORS

Constructors

📌 WHAT	A special method that runs automatically when an object is created. Used to initialize (set initial values) the object.

❓ WHY	To set default values when object is created. Like when you buy a new phone — it comes pre-configured with some settings.

💻 Code Example:

```
class Student {
    String name;
    int age;

    // Default Constructor (no parameters)
    Student() {
        name = "Unknown";
        age = 0;
        System.out.println("Object created!");
    }

    // Parameterized Constructor
    Student(String n, int a) {
        name = n;
        age = a;
    }

    // Copy Constructor
    Student(Student s) {
        name = s.name;
        age = s.age;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();           // Default constructor
        Student s2 = new Student("Rahul", 20); // Parameterized
        Student s3 = new Student(s2);          // Copy constructor
        System.out.println(s2.name);  // Rahul
        System.out.println(s3.name);  // Rahul (copy of s2)
```

```
        }
    }
```

🎯 Interview Questions:

Q17. **What is a constructor?**

→ Special method with same name as class, no return type. Automatically called when object is created. Used to initialize object.

Q18. **What is constructor overloading?**

→ Having multiple constructors with different parameters. Java decides which one to call based on arguments passed.

Q19. **What is the difference between constructor and method?**

→ Constructor: same name as class, no return type, called automatically on object creation. Method: has return type, called explicitly.

Q20. **What is a default constructor?**

→ Constructor with no parameters. If you don't write any constructor, Java provides one automatically (implicit default constructor).

Q21. **What is 'this' keyword in constructor?**

→ 'this' refers to the current object. this.name = name avoids confusion between instance variable and parameter with same name.

⚠️ Common Mistakes:

- Giving a return type to constructor (even void) — it becomes a method, not constructor
- Forgetting 'this' keyword when parameter and variable have same name
- Not calling super() first in child constructor when needed

🔄 CHAPTER 4: METHOD OVERLOADING & OVERRIDING

Method Overloading & Overriding

Method Overloading (Compile-time Polymorphism)

📌 WHAT	Same method name, different parameters. Java decides which one to call based on arguments — at COMPILE TIME.
🌍 Example	A calculator's add() method — can add 2 integers, 3 integers, or 2 decimals.

```
class Calculator {
    int add(int a, int b) { return a + b; }              // 2 ints
    int add(int a, int b, int c) { return a + b + c; }  // 3 ints
    double add(double a, double b) { return a + b; }    // 2 doubles
}

public class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println(c.add(2, 3));       // 5
        System.out.println(c.add(1, 2, 3));    // 6
        System.out.println(c.add(1.5, 2.5));   // 4.0
    }
}
```

Method Overriding (Runtime Polymorphism)

📌 WHAT	Child class provides its own implementation of a method already defined in parent class. Same name, same parameters.

```
class Animal {
    void sound() { System.out.println("Animal makes sound"); }
}

class Dog extends Animal {
    @Override
    void sound() { System.out.println("Dog says: Woof! 🐕"); }
}

class Cat extends Animal {
    @Override
    void sound() { System.out.println("Cat says: Meow! 🐈"); }
}
```

🎯 Key Differences (Interview Favorite!):

Feature	Overloading	Overriding
Class	Same class	Parent & Child class
Parameters	Must be different	Must be same
Return type	Can differ	Must be same (or covariant)
Binding	Compile-time (early)	Runtime (late)
Keyword	No keyword needed	@Override (best practice)
Inheritance	Not required	Required
Polymorphism type	Compile-time	Runtime

🎯 Interview Questions:

Q22. **Can we overload the main method?**

→ Yes! You can have multiple main methods with different parameters. But JVM calls only main(String[] args) to start.

Q23. **Can we override static methods?**

→ No! Static methods belong to class, not object. They are hidden, not overridden. This is called Method Hiding.

Q24. **Can we override private methods?**

→ No! Private methods are not visible to child class, so they cannot be overridden.

🔐 CHAPTER 5: ACCESS MODIFIERS

Access Modifiers

📌 WHAT	Keywords that control who can access class, variable, or method. Like security levels — public park vs private property.

Modifier	Same Class	Same Package	Child Class	Other Package
private	✅ Yes	❌ No	❌ No	❌ No
default (no keyword)	✅ Yes	✅ Yes	❌ No	❌ No
protected	✅ Yes	✅ Yes	✅ Yes	❌ No
public	✅ Yes	✅ Yes	✅ Yes	✅ Yes

```
public class MyClass {
    private int a = 1;     // only inside MyClass
    int b = 2;             // default: same package
    protected int c = 3;   // same package + child classes
    public int d = 4;      // everywhere
}
```

🎯 Interview Questions:

Q25. **What is the default access modifier?**
→ When no modifier is written, it is 'package-private' (default). Accessible only within the same package.

Q26. **What is the most restrictive access modifier?**
→ private — accessible only within the same class.

Q27. **Can we declare a class as private?**
→ No! Top-level class can be public or default only. Inner classes can be private.

⚡ CHAPTER 6: STATIC KEYWORD

Static Keyword

📌 WHAT	Static means 'belonging to the class, not to any object'. Shared among all objects of the class.
🌍 Example	School name is same for all students — it's a static property. But each student has their own name (instance variable).

```java
class Student {
    String name;            // instance variable — different for each student
    static String school = "ABC School"; // static — shared by all
    static int count = 0;   // counts total students

    Student(String name) {
        this.name = name;
        count++;            // increments shared counter
    }

    static void showSchool() {  // static method
        System.out.println("School: " + school);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Rahul");
        Student s2 = new Student("Priya");
        System.out.println(Student.count);   // 2 (not s1.count)
        Student.showSchool();                // School: ABC School
    }
}
```

🎯 Interview Questions:

Q28. **Can static method access instance variable?**

→ No! Static method runs without any object, so it can't access instance variables (which belong to objects).

Q29. **Why is main() method static?**

→ JVM calls main() before any object is created. Static allows calling without creating object.

Q30. **What is static block?**

→ Code block with 'static' keyword that runs once when class is loaded into memory. Used for static initialization.

🔒 CHAPTER 7: FINAL KEYWORD

Final Keyword

📌 WHAT	'final' means it CANNOT be changed. Final variable = constant. Final method = cannot override. Final class = cannot inherit.

```
final int MAX = 100;      // final variable — constant
// MAX = 200;             // ❌ Error! Cannot reassign final

class Parent {
    final void show() { System.out.println("Parent show"); }
    // final method — cannot be overridden
}

final class ImmutableClass { }  // final class — cannot be extended
// class Child extends ImmutableClass { } // ❌ Error!

// String is a famous example of final class in Java!
// String str = "hello";  — cannot extend String
```

🎯 Interview Questions:

Q31. **What is the difference between final, finally, and finalize?**

→ final: keyword for constant/non-overridable. finally: block in exception handling, always executes. finalize(): method called by GC before object is destroyed.

Q32. **Can final variable be initialized later?**

→ Yes, only once! Called blank final variable. Must be initialized in constructor.

Q33. **Why is String class final?**

→ To make strings immutable and secure. If someone could extend String, they could change its behavior, breaking security.

🔤 CHAPTER 8: STRING & STRINGBUILDER

String & StringBuilder

String

📌 WHAT	String is immutable — once created, cannot be changed. Any modification creates a NEW string object in memory.

```java
String s1 = "Hello";
String s2 = "Hello";       // same object from String Pool!
String s3 = new String("Hello");  // new object in Heap

System.out.println(s1 == s2);      // true (same reference in pool)
System.out.println(s1 == s3);      // false (different objects)
System.out.println(s1.equals(s3)); // true (same content)

// Common String Methods
String str = "Hello World";
System.out.println(str.length());           // 11
System.out.println(str.toUpperCase());      // HELLO WORLD
System.out.println(str.substring(0, 5));    // Hello
System.out.println(str.contains("World")); // true
System.out.println(str.replace("World","Java")); // Hello Java
System.out.println(str.trim());             // removes spaces
System.out.println(str.charAt(0));          // H
```

StringBuilder

📌 **WHAT** Mutable version of String. Can be modified without creating new objects. Much faster for multiple concatenations.

```java
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");        // adds to existing
sb.insert(5, ",");          // inserts at position 5
sb.delete(5, 6);             // deletes characters
sb.reverse();                // reverses string
sb.replace(0, 5, "Hi");     // replaces range
System.out.println(sb.toString());  // convert to String
```

🎯 Interview Questions:

Q34. **What is the difference between String and StringBuilder?**

→ String is immutable — every change creates new object. StringBuilder is mutable — modifies same object. StringBuilder is faster for loops.

Q35. **What is String Pool?**

→ A special area in heap where Java stores String literals. If same string exists, it reuses it (saves memory). new String() bypasses pool.

Q36. **== vs .equals() for String?**

→ == compares references (memory addresses). .equals() compares actual content. Always use .equals() for string comparison.

Q37. **What is StringBuffer vs StringBuilder?**

→ StringBuffer is thread-safe (synchronized) but slower. StringBuilder is not thread-safe but faster. For single-thread programs, prefer StringBuilder.

📊 CHAPTER 9: ARRAYS

Arrays

📌 WHAT	A collection of fixed-size elements of the same type, stored in contiguous memory locations. Index starts from 0.

```
// 1D Array
int[] marks = new int[5];        // declaration with size
int[] scores = {90, 85, 78, 92, 88};  // with values

System.out.println(scores[0]);   // 90 (first element)
System.out.println(scores.length); // 5 (total elements)

// Loop through array
for (int i = 0; i < scores.length; i++) {
    System.out.println(scores[i]);
}

// Enhanced for loop (for-each)
for (int score : scores) {
    System.out.println(score);
}

// 2D Array (like table)
int[][] matrix = {{1,2,3}, {4,5,6}, {7,8,9}};
System.out.println(matrix[1][2]); // 6 (row 1, col 2)

// Sorting array
import java.util.Arrays;
Arrays.sort(scores);
System.out.println(Arrays.toString(scores));  // [78, 85, 88, 90, 92]
```

🎯 Interview Questions:

Q38. **What is the default value of array elements?**

→ int/long: 0, double/float: 0.0, boolean: false, String/Object: null

Q39. **What happens if we access out-of-bound index?**

→ Java throws ArrayIndexOutOfBoundsException at runtime.

Q40. **What is the difference between Array and ArrayList?**

→ Array: fixed size, can store primitives. ArrayList: dynamic size, stores only objects. Array is faster, ArrayList is flexible.

⚠️ CHAPTER 10: EXCEPTION HANDLING

Exception Handling

📌 WHAT	A way to handle runtime errors gracefully so the program doesn't crash. Exceptions are unexpected events during execution.
🌍 Example	Like an ATM — if you enter wrong PIN, it shows error message but doesn't break. That's exception handling.

```
public class Main {
    public static void main(String[] args) {
        try {
            // Code that might throw exception
            int a = 10 / 0;  // ArithmeticException!
            System.out.println("This won't print");
        }
        catch (ArithmeticException e) {
            // Handle the exception
            System.out.println("Error: " + e.getMessage());
        }
        catch (Exception e) {
            // Catch any other exception
            System.out.println("General error: " + e.getMessage());
        }
        finally {
            // Always runs — cleanup code
            System.out.println("Finally block always runs!");
        }
        System.out.println("Program continues normally");
    }
}
```

Common Exception Types:

Exception	When It Occurs
NullPointerException	Calling method on null object (String s = null; s.length())
ArrayIndexOutOfBoundsException	Accessing index that doesn't exist (arr[10] when size is 5)
ArithmeticException	Dividing by zero (10/0)
ClassCastException	Wrong type casting
NumberFormatException	Converting invalid string to number (Integer.parseInt("abc"))
StackOverflowError	Infinite recursion

Custom Exception:

```
// Creating your own exception
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String msg) {
        super(msg);
    }
}
```

```java
class BankAccount {
    double balance = 500;

    void withdraw(double amount) throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException("Not enough balance!");
        }
        balance -= amount;
    }
}
```

🎯 Interview Questions:

Q41. **What is the difference between Checked and Unchecked exceptions?**

→ Checked: must be handled at compile time (IOException, SQLException). Unchecked: occur at runtime, not mandatory to handle (NullPointerException, ArithmeticException). Unchecked extends RuntimeException.

Q42. **What is the difference between throw and throws?**

→ 'throw' is used to manually throw an exception. 'throws' in method signature declares that method might throw exception. throw is inside method body, throws is in method declaration.

Q43. **Can finally block be skipped?**

→ Yes, only if System.exit() is called before finally, or JVM crashes.

Q44. **What is the difference between Error and Exception?**

→ Error: serious problems (OutOfMemoryError, StackOverflowError) — cannot be recovered. Exception: expected problems that can be handled in code.

📚 CHAPTER 11: COLLECTIONS FRAMEWORK

Collections Framework

📌 WHAT	A set of classes and interfaces to store, manage, and manipulate groups of objects. Much more flexible than arrays.

1. List (Ordered, allows duplicates)

```
import java.util.*;

// ArrayList — fast access, slow insert/delete in middle
ArrayList<String> list = new ArrayList<>();
list.add("Apple");
list.add("Banana");
list.add("Apple");  // duplicates allowed ✅
list.add(1, "Mango"); // insert at index 1
list.remove("Banana");
System.out.println(list.get(0));  // Apple
System.out.println(list.size());  // 3
System.out.println(list.contains("Apple")); // true

// Iterating
for (String fruit : list) {
    System.out.println(fruit);
}

// LinkedList — fast insert/delete, slow access
LinkedList<Integer> ll = new LinkedList<>();
ll.addFirst(10);
ll.addLast(20);
```

2. Set (No duplicates, unordered)

```
// HashSet — no duplicates, no order
HashSet<String> set = new HashSet<>();
set.add("Apple");
set.add("Banana");
set.add("Apple");  // duplicate — ignored ❌
System.out.println(set.size());  // 2

// TreeSet — sorted order automatically
TreeSet<Integer> ts = new TreeSet<>();
ts.add(30); ts.add(10); ts.add(20);
System.out.println(ts);  // [10, 20, 30] — sorted!

// LinkedHashSet — maintains insertion order
LinkedHashSet<String> lhs = new LinkedHashSet<>();
lhs.add("Banana"); lhs.add("Apple"); lhs.add("Cherry");
```

```
System.out.println(lhs);  // [Banana, Apple, Cherry] — insertion order
```

3. Map (Key-Value pairs)

```
// HashMap — key-value, no order
HashMap<String, Integer> map = new HashMap<>();
map.put("Rahul", 90);
map.put("Priya", 85);
map.put("Rahul", 95);  // duplicate key — value updated!

System.out.println(map.get("Rahul"));     // 95
System.out.println(map.containsKey("Priya")); // true
System.out.println(map.size());            // 2

// Iterating Map
for (Map.Entry<String,Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " : " + entry.getValue());
}

// TreeMap — sorted by key
TreeMap<String,Integer> tm = new TreeMap<>(map);
System.out.println(tm);  // sorted alphabetically by name
```

🎯 Interview Questions:

Q45. **What is the difference between List and Set?**

→ List: ordered, allows duplicates. Set: unordered (HashSet), no duplicates. List maintains insertion order, Set doesn't guarantee order.

Q46. **How does HashMap work internally?**

→ Uses array of buckets + linked list/tree. Key's hashCode() determines bucket. equals() checks for same key. Since Java 8, bucket with 8+ entries converts to Red-Black tree.

Q47. **What is the difference between HashMap and HashSet?**

→ HashMap stores key-value pairs. HashSet stores only values. HashSet internally uses HashMap with dummy values.

Q48. **What is ConcurrentHashMap?**

→ Thread-safe version of HashMap. Instead of synchronizing entire map, it locks only part of the map (segment/bucket level). Better performance than Hashtable.

🧵 CHAPTER 12: MULTITHREADING (BASICS)

Multithreading

📌 WHAT	Running multiple threads simultaneously. Thread = smallest unit of execution. Multithreading = multiple tasks running at the same time.
🌍 Example	Browser downloads a file AND plays music at the same time. Each is a separate thread.

💻 Two ways to create threads:

```
// WAY 1: Extend Thread class
class MyThread extends Thread {
    public void run() {  // thread logic goes here
        for (int i = 1; i <= 3; i++) {
            System.out.println("Thread: " + i);
        }
    }
}

// WAY 2: Implement Runnable interface (Preferred!)
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable running!");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();  // starts new thread — calls run() internally

        Thread t2 = new Thread(new MyRunnable());
        t2.start();

        // Using Lambda (Java 8+)
        Thread t3 = new Thread(() -> System.out.println("Lambda thread!"));
        t3.start();
    }
}
```

Thread Lifecycle:

State	Meaning
New	Thread created but not started yet
Runnable	Thread is ready to run (waiting for CPU)
Running	Thread is currently executing

State	Meaning
Blocked/Waiting	Thread is waiting for resource or lock
Terminated	Thread has finished execution

🎯 Interview Questions:

Q49. **What is the difference between Thread and Runnable?**

→ Thread: extending class, no multiple inheritance possible. Runnable: implementing interface, allows extending another class too. Runnable is preferred.

Q50. **What is synchronized keyword?**

→ Prevents two threads from accessing same code block at same time. Only one thread can execute synchronized block — prevents data corruption.

Q51. **What is deadlock?**

→ Two threads waiting for each other to release a resource — both wait forever. Like two people at a narrow bridge, each waiting for other to move.

Q52. **Difference between start() and run()?**

→ start() creates new thread and calls run(). If you call run() directly, it runs in current thread (no new thread created) — not multithreading!

📁 CHAPTER 13: FILE HANDLING (BASICS)

File Handling

📌 WHAT	Reading from and writing to files stored on disk. Java provides File, FileWriter, FileReader, BufferedReader classes for this.

```
import java.io.*;

public class FileExample {
    public static void main(String[] args) {

        // WRITING to a file
        try {
            FileWriter fw = new FileWriter("test.txt");
            fw.write("Hello, File Handling!\n");
            fw.write("Second line");
            fw.close();  // always close!
            System.out.println("File written successfully!");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }

        // READING from a file
        try {
            BufferedReader br = new BufferedReader(new FileReader("test.txt"));
```

```
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
            br.close();
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }

        // Try-with-resources (auto-closes file — Java 7+)
        try (BufferedReader br = new BufferedReader(new FileReader("test.txt")))
{
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) { }
    }
}
```

🎯 Interview Questions:

Q53. **What is the difference between FileWriter and BufferedWriter?**

→ FileWriter writes character by character directly to file — slow. BufferedWriter writes to a buffer first, then flushes to file at once — faster.

Q54. **What is try-with-resources?**

→ Java 7 feature. Resources (files, connections) are automatically closed after try block — no need to call close() manually. Cleaner code.

Q55. **What is serialization?**

→ Converting object to byte stream to save to file or send over network. Class must implement Serializable interface. Deserialization is reverse — bytes back to object.

🎯 IMPORTANT INTERVIEW QUESTIONS (MIXED)

Top Java Interview Questions with Answers

Q56. **What is JDK, JRE, and JVM?**

→ JVM (Java Virtual Machine): runs bytecode. JRE (Runtime Environment) = JVM + libraries (to run Java programs). JDK (Development Kit) = JRE + compiler + tools (to develop Java programs).

Q57. **Is Java platform-independent? How?**

→ Yes! Java code compiles to bytecode (.class file). JVM installed on any OS can run this bytecode. 'Write Once, Run Anywhere.'

Q58. **What is the difference between == and equals()?**

→ == compares references (memory addresses). equals() compares content/values. For objects, always use equals(). For primitives, use ==.

Q59. **What is autoboxing and unboxing?**

→ Autoboxing: automatic conversion of primitive to wrapper (int → Integer). Unboxing: wrapper to primitive (Integer → int). Java does this automatically.

Q60. **What are wrapper classes?**

→ Classes for primitive types: int→Integer, double→Double, char→Character. Needed for Collections (which require objects, not primitives).

Q61. **What is the difference between ArrayList and Array?**

→ Array: fixed size, can store primitives, faster. ArrayList: dynamic size, stores only objects, more flexible methods.

Q62. **What is an Iterator?**

→ An object to traverse a collection. hasNext() checks if element exists, next() returns next element. Safer than for loop for removal during iteration.

Q63. **What is the difference between Iterator and ListIterator?**

→ Iterator: forward only. ListIterator: forward and backward, can add/set elements. ListIterator works only with List.

Q64. **What is garbage collection in Java?**

→ Automatic memory management. JVM automatically removes objects that are no longer referenced. Programmer doesn't need to manually free memory (unlike C/C++).

Q65. **What is the difference between Heap and Stack?**

→ Stack: stores local variables, method calls — LIFO order, faster. Heap: stores objects — shared memory, managed by GC. Stack is per thread, Heap is shared.

Q66. **What is an interface in Java?**

→ Blueprint with abstract methods. Class uses 'implements'. Supports multiple implementation. Before Java 8: only abstract methods. Java 8+: can have default and static methods.

Q67. **What is the difference between abstract class and interface?**

→ Abstract class: can have constructor, concrete methods, any modifier, single inheritance. Interface: no constructor, all methods public, multiple implementation possible.

Q68. **What is a lambda expression?**

→ Short way to write anonymous functions (Java 8+). Instead of anonymous class, write concise code: (a,b) -> a+b. Works with functional interfaces (interfaces with one abstract method).

Q69. **What is Java 8's Stream API?**

→ Processes collections in functional style. filter(), map(), reduce(), collect() etc. Supports pipeline operations. Can be parallel for performance.

Q70. **What is the difference between Comparable and Comparator?**

→ Comparable: class sorts itself (compareTo() in same class). Comparator: separate class/lambda defines custom sort. Comparator is more flexible — can have multiple sort logics.

Q71. **What is Optional in Java 8?**

→ A container for value that might be null. Avoids NullPointerException. Optional.of(), Optional.empty(), isPresent(), orElse() methods.

Q72. **What is immutability? Give example.**

→ Object whose state cannot change after creation. String is immutable. Create immutable class: final class, private fields, no setters, final fields.

Q73. **What is the difference between shallow copy and deep copy?**

→ Shallow copy: copies references (both point to same nested objects). Deep copy: copies everything recursively (completely independent). Use clone() or serialization for deep copy.

Q74. **What is a functional interface?**

→ Interface with exactly ONE abstract method. Can use lambda expressions with it. Examples: Runnable, Comparator, Callable. @FunctionalInterface annotation used.

Q75. **What is the difference between HashMap and LinkedHashMap?**

→ HashMap: no order guarantee. LinkedHashMap: maintains insertion order. LinkedHashMap slightly slower due to maintaining a doubly-linked list.

📊 DATA STRUCTURE DIFFERENCES (Simple Words)

Difference Between Data Structures

1. Array vs ArrayList

Feature	Array	ArrayList
Size	Fixed (cannot change)	Dynamic (grows/shrinks)
Primitives	Can store (int, double...)	Only objects (Integer, Double...)
Performance	Faster (direct memory)	Slightly slower (overhead)
Methods	No built-in methods	add(), remove(), contains() etc.
Syntax	int[] arr = new int[5]	ArrayList<Integer> list = new ArrayList<>()
Multi-dim	Supported easily	Need List of List
When to use?	Fixed size, performance needed	Size unknown, need flexibility

2. ArrayList vs LinkedList

Feature	ArrayList	LinkedList
Internal structure	Dynamic array	Doubly linked list
Access (get by index)	O(1) — very fast	O(n) — slow (traverse)
Insert/Delete (middle)	O(n) — slow (shift elements)	O(1) — fast (just change pointers)
Memory	Less (only data)	More (data + prev + next pointers)
Iteration	Faster	Slightly slower
When to use?	Read a lot, less modification	Insert/delete a lot, less reading

3. List vs Set

Feature	List	Set
Duplicates	✅ Allowed	❌ Not allowed
Order	Maintains insertion order	No guaranteed order (HashSet)
Null values	Multiple nulls allowed	Only one null (HashSet)
Access by index	✅ Yes (get(0))	❌ No index

Feature	List	Set
Implementations	ArrayList, LinkedList	HashSet, TreeSet, LinkedHashSet
When to use?	Need order, allow duplicates	Need unique elements only

4. HashSet vs TreeSet

Feature	HashSet	TreeSet
Order	No guaranteed order	Sorted ascending order always
Performance	O(1) — constant time	O(log n) — tree operations
Null values	One null allowed	No null allowed (throws exception)
Internal structure	HashMap (hash table)	Red-Black Tree
Custom sort?	Not possible	Yes, with Comparator
When to use?	Fast lookup, order not needed	Need sorted unique elements

5. HashMap vs Hashtable

Feature	HashMap	Hashtable
Thread safety	❌ Not synchronized	✅ Synchronized (thread-safe)
Performance	Faster	Slower (due to sync overhead)
Null keys	One null key, many null values	No null key or value
Introduced in	Java 1.2 (Collections)	Java 1.0 (Legacy class)
Iterator	Fail-fast iterator	Enumerator (not fail-fast)
Recommended?	Yes (use ConcurrentHashMap for threads)	No (legacy — avoid)
When to use?	Single-threaded programs	Avoid — use ConcurrentHashMap

6. Quick Summary Table

Collection	Order?	Duplicates?	Null?	Thread-safe?	Best For
ArrayList	Yes	Yes	Yes	No	Fast read
LinkedList	Yes	Yes	Yes	No	Fast insert/delete
HashSet	No	No	1 null	No	Unique, fast lookup
TreeSet	Sorted	No	No	No	Sorted unique values

Collection	Order?	Duplicates?	Null?	Thread-safe?	Best For
LinkedHashSet	Insertion	No	1 null	No	Ordered unique
HashMap	No	Keys: No	1 null key	No	Key-value, fast
TreeMap	Sorted	Keys: No	No	No	Sorted key-value
Hashtable	No	Keys: No	No	Yes	Legacy (avoid)
ConcurrentHashMap	No	Keys: No	No	Yes	Thread-safe map

🎉 All the best for your Java Interview! 🎉

Remember: Practice code daily + Revise notes before interview!